Ψηφιακή Επεξεργασία Εικόνας

Εργασία 2

Ανίχνευση Ακμών και Κύκλων

Τμήμα Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών ΑΠΘ

8ο εξάμηνο

Θεσσαλονίκη, Μάιος 2025

Υπεύθυνη εργασίας: Μανωλίδου Ανατολή

Email: amanolid@ece.auth.gr

ΑΕΜ: 10874

Περιεχόμενα

1 Calculation of Convolution……..…….……3

1.1 fir_conv.py……..……..……....…3

2 Edge Detection……..……..……4

2.2 sobel_edge.py……..……..….…....4

2.2 log_edge.py……..…….……..5

3 Circle Detection……..……7

3.1 circ_hough.py…...……..…….……7

4 Τελικά Αποτελέσματα και Συμπεράσματα……10

5.1 Edge Detection……10

5.2 Circle Detection……...13

1 Calculation of Convolution

Θεμελιώδες κομμάτι της συγκεκριμένης εργασίας αποτελεί ο υπολογισμός της συνέλιξης μιας εικόνας εισόδου με μια συνελικτική μάσκα.

1.1 fir_conv.py

Παρακάτω παρατίθεται το κομμάτι του κώδικα στο υλοποιείται η συνάρτηση **fir_conv.py:**

```
def fir_conv(
        in_img_array: np.ndarray,
        h: np.ndarray,
) -> np.ndarray:

        in_height, in_width = in_img_array.shape
        mask_height, mask_width = h.shape

        pad_height = mask_height // 2
        pad_width = mask_width // 2

        padded_img = np.pad(in_img_array, ((pad_height, pad_height), (pad_width,
pad_width)), mode='constant', constant_values=0)

        out_img_array = np.zeros_like(in_img_array)

        h = np.flipud(np.fliplr(h))

        for i in range(in_height):
                for j in range(in_width):
                        out_img_array[i, j] = np.sum(h * padded_img[i:i +
mask_height, j:j + mask_width])

        return out_img_array
```

Αρχικά έχουν οριστεί οι είσοδοι και οι έξοδοι της συνάρτησης όπως αυτά δόθηκαν στην εκφώνηση της άσκησης και συγκεκριμένα έχω επιλέξει την υλοποίηση δίχως τον ορισμό των πινάκων για την αποθήκευση των θέσεων της αρχής των αξόνων . Στη συνέχεια, κάνουμε zero padding στην εικόνα μας, ώστε να μπορέσουμε να “περάσουμε” τη μάσκα πάνω από την εικόνα χωρίς να χρειαστεί να βγει εκτός ορίων. Επιπλέον κάνουμε flip τη μάσκα μας και τέλος, με ένα διπλό for loop περνάμε από όλα τα στοιχεία της εικόνας και εφαρμόζουμε το τύπο της συνέλιξης:

$$\sum_{k1=N_1,min}^{N_{1,max}} \sum_{k2=N_2,min}^{N_{2,max}} h(k_1,k_2) f(n_1-k_1, n_2-k_2)$$

2 Edge Detection

2.1 sobel_edge.py

Παρακάτω παρατίθεται το κομμάτι του κώδικα στο υλοποιείται η συνάρτηση **sobel_edge.py**:

```
def sobel_edge(
        in_img_array: np.ndarray,
        thres: float
) -> np.ndarray:

        sobel_x = np.array([[-1, 0, 1],
                            [-2, 0, 2],
                            [-1, 0, 1]])

        sobel_y = np.array([[1, 2, 1],
                            [0, 0, 0],
                            [-1, -2, -1]])

        grad_x = CONV.fir_conv(in_img_array, sobel_x)

        grad_y = CONV.fir_conv(in_img_array, sobel_y)

        grad_magnitude = np.sqrt(grad_x**2 + grad_y**2)

        grad_magnitude = grad_magnitude / np.max(grad_magnitude)

        edge_img = np.where(grad_magnitude > thres, 1, 0)

        return edge_img
```

Αρχικά έχουν οριστεί οι είσοδοι και οι έξοδοι της συνάρτησης όπως αυτά δόθηκαν στην εκφώνηση της άσκησης, . Έπειτα, ορίζουμε τις δυο Sobel μάσκες, μια για την κατεύθυνση των x και άλλη μια για των y. Με σκοπό να προσεγγίσουμε τις παραγώγους της εικόνας εφαρμόζουμε τη συνέλιξη της εικόνας με τις Sobel μάσκες. Στη συνέχεια, υπολογίζουμε το μέτρο της κλίσης για

το κάθε σημείο και το κανονικοποιούμε ώστε να παίρνουμε τιμές στο εύρος [0,1]. Τέλος, δημιουργούμε τη binary εικόνα εξόδου μας, βάζοντας τη τιμή της μονάδας σε όποια σημεία έχουν μέτρο κλίσης μεγαλύτερο από το threshold, δηλαδή τα θεωρούμε ως pixel ακμής και τη τιμή του μηδενός σε όποια σημεία δε το ξεπερνάνε.

2.2 log_edge.py

Παρακάτω παρατίθεται το κομμάτι του κώδικα στο υλοποιείται η συνάρτηση **log_edge.py**:

```
def log_edge(in_img_array: np.ndarray) -> np.ndarray:

    log_mask = np.array([[0, 0, -1, 0, 0],
                         [0, -1, -2, -1, 0],
                         [-1, -2, 16, -2, -1],
                         [0, -1, -2, -1, 0],
                         [0, 0, -1, 0, 0]])


    smoothed_img = gaussian_filter(in_img_array, sigma=1.5)

    log_img = CONV.fir_conv(smoothed_img, log_mask)

    edge_img = np.zeros_like(log_img)

    threshold = 0.2
    for i in range(1, log_img.shape[0] - 1):
        for j in range(1, log_img.shape[1] - 1):
            patch = log_img[i-1:i+2, j-1:j+2]
            center = patch[1, 1]
            neighbors = [
                patch[0, 0], patch[0, 1], patch[0, 2],
                patch[1, 0],             patch[1, 2],
                patch[2, 0], patch[2, 1], patch[2, 2]
            ]

            opposite_signs = sum(
                1 for neighbor in neighbors
                if np.sign(center) != np.sign(neighbor) and abs(center -
neighbor) > threshold
            )

            if opposite_signs >= 2:
```

```
                edge_img[i, j] = 1
                continue

            diagonal_pairs = [
                (patch[0, 0], patch[1, 1]),
                (patch[0, 2], patch[1, 1]),
                (patch[2, 0], patch[1, 1]),
                (patch[2, 2], patch[1, 1]),
                (patch[0, 0], patch[2, 2]),
                (patch[0, 2], patch[2, 0])
            ]

            for a, b in diagonal_pairs:
                if np.sign(a) != np.sign(b) and abs(a - b) > threshold:
                    edge_img[i, j] = 1
                    break

    return edge_img
```

Αρχικά έχουν οριστεί οι είσοδοι, οι έξοδοι της συνάρτησης και η διακριτή μάσκα όπως αυτά δόθηκαν στην εκφώνηση της άσκησης. Στη συνέχεια, εξομαλύνουμε την εικόνα με τη χρήση ενός Gaussian φίλτρου (επιλέχθηκε τυπική απόκλιση ίση με 1.5) και για να προσεγγίσουμε τη παράγωγο δεύτερης τάξης της εικόνας εφαρμόζουμε τη συνέλιξη της εξομαλυμένης εικόνας με τη μάσκα που έχουμε ορίσει.

Έπειτα, ακολουθεί η λογική του zero-crossing για την ανίχνευση των ακμών και πιο συγκεκριμένα ξεκινάμε να διατρέχουμε από όλα τα στοιχεία της εικόνας (εκτός από το περίγραμμα της ώστε να μη βγούμε εκτός ορίων) και γύρω από κάθε στοιχείο δημιουργούμε ένα "παραθυράκι" (στο οποίο αναφερόμαστε ως patch) διαστάσεων $3x3$. Για να θεωρηθεί μια ακμή ακολούθησα δυο τακτικές, όσο αφορά τη πρώτη, υπολογίζουμε το γινόμενο μεταξύ του κέντρου του patch με όλα τα υπόλοιπα 8 γειτονικά του στοιχεία και για κάθε γινόμενο ελέγχουμε αν η "διαφορά" στη φωτεινότητά τους ξεπερνάει το threshold μας (η προσθήκη του threshold έγινε με τη λογική να μπορέσουμε να αποφύγουμε μικρές μεταβολές που μπορεί να προέκυψαν από θόρυβο). Έπειτα, κρατάμε το πλήθος των γινομένων για τα οποία έχουμε αρνητικό πρόσημο και διαφορά που ξεπερνάει το threshold και θεωρούμε πως εάν αυτό το πλήθος ξεπερνάει τη τιμή 1 έχουμε ακμή και προχωράμε στο επόμενο pixel. Σε περίπτωση που το πλήθος δε ξεπεράσει τη τιμή 2, προχωράμε στη δεύτερη τακτική όπου αντί να συγκρίνουμε το κεντρικό στοιχείο του patch με τους γύρω γείτονες του εστιάζουμε στη σχέση ανάμεσα σε διαγώνια ζεύγη pixel. Βέβαια στη συγκεκριμένη περίπτωση που έχουμε patch $3x3$, έχουμε δυο μόνο επιπλέον ζεύγη που δεν έχουν ελεγχθεί και στη πρώτη τακτική (το επάνω αριστερά με το κάτω δεξιά και επάνω δεξιά με το κάτω αριστερά). Όπως και πριν για κάθε ένα από αυτά τα ζεύγη, ελέγχουμε αν τα δύο στοιχεία έχουν αντίθετο πρόσημο και αν η διαφορά των τιμών τους ξεπερνά το καθορισμένο threshold. Αν έστω

και ένα από τα ζεύγη ικανοποιεί αυτή τη συνθήκη, το σημείο αυτό χαρακτηρίζεται ως ακμή και η τιμή του εξόδου στην αντίστοιχη θέση ορίζεται ως μονάδα.

3 Circle Detection

3.1 circ_hough.py

Παρακάτω παρατίθεται το κομμάτι του κώδικα στο υλοποιείται η συνάρτηση **circ_hough.py**:

```
def circ_hough(
        in_img_array: np.ndarray,
        R_min: float,
        R_max: float,
        dim: np.ndarray,
        V_min: int
) -> tuple[np.ndarray, np.ndarray, np.ndarray]:

        hough_space = np.zeros((dim[0], dim[1], dim[2]))

        a_inter = int(in_img_array.shape[0] / dim[0])
        b_inter = int(in_img_array.shape[1] / dim[1])
        r_inter = int((R_max - R_min) / dim[2])

        ys, xs = np.nonzero(in_img_array)
        r_vals = np.linspace(R_min, R_max, dim[2])

        for y, x in zip(ys, xs):
                for r in (r_vals):
                        for theta in range(0, 360, 2):
                                a = int(x - r * np.cos(theta * np.pi / 180))
                                b = int(y - r * np.sin(theta * np.pi / 180))

                                a_bin = int(a / a_inter)
                                b_bin = int(b / b_inter)
                                r_bin = int((r - R_min) / r_inter)

                                if 0 <= a_bin < dim[0] and 0 <= b_bin < dim[1]
and 0 <= r_bin < dim[2]:
                                        hough_space[a_bin, b_bin, r_bin] += 1

        centers = []
```

```
    radii = []
    votes = []

    for i in range(hough_space.shape[0]):
            for j in range(hough_space.shape[1]):
                    for k in range(hough_space.shape[2]):
                            if hough_space[i, j, k] > V_min:


                                    ai = i * a_inter
                                    bi = j * b_inter
                                    ri = R_min + (k * r_inter)


                                    aii = (i + 1) * a_inter
                                    bii = (j + 1) * b_inter
                                    rii = R_min + ((k + 1) * r_inter)

                                    cx = (ai + aii) / 2
                                    cy = (bi + bii) / 2
                                    r = (ri + rii) / 2

                                    centers.append([cx, cy])
                                    radii.append(r)
                                    votes.append(hough_space[i, j, k])

    centers = np.array(centers)
    radii = np.array(radii)
    votes = np.array(votes)

    return centers, radii, votes
```

Αρχικά έχουν οριστεί οι είσοδοι και οι έξοδοι της συνάρτησης με μια μικρή παρέμβαση στην δοσμένη εκδοχή καθώς έχει προστεθεί και μια μεταβλητή εισόδου *R_min* που ορίζει την ελάχιστη τιμή για το εύρος αναζήτησης ακτίνας και μια ακόμη μεταβλητή εξόδου η *votes* που αντιπροσωπεύει το πλήθος των votes που έχει κάθε εντοπισμένος κύκλος (η προσθήκη αυτή έγινε για να βρεθεί πιο εύκολα το κατάλληλο *V_min*).

Αρχικά, ορίζουμε τον χώρο Hough στον οποίο και θα τοποθετήσουμε αργότερα τις ψήφους. Στη συνέχεια, υπολογίζουμε το εύρος των διαστημάτων για τα a, b και r και μέσω της $np.linspace()$ δημιουργούμε τυχαίες τιμές (τόσες όσες και το πλήθος των διαστημάτων για το r που περνάμε ως όρισμα στη συνάρτηση) για την ακτίνα r στο διάστημα $[R_{min}, R_{max}]$. Έπειτα, μέσω ενός διπλού for loop διατρέχουμε απ' όλα τα στοιχεία της εικόνας τα οποία θεωρούνται ως ακμή και για κάθε μια ακτίνα έχουμε ως εξής:

Υπολογίζουμε τα a και b μέσω των γνωστών τύπων:

$$a = x - r * \cos(\theta) \text{ και } b = y - r * sin(\theta)$$

όπου ως γωνία θ θεωρούμε όλες τις γωνίες από 0 έως 2π (συγκεκριμένα έχει επιλεχθεί βήμα ίσο με 2 ώστε να έχουμε λίγη λιγότερη πολυπλοκότητα δίχως να χάνουμε αρκετή πληροφορία). Έπειτα, βρίσκουμε σε ποιο bin του διαστήματος ανήκουν οι τιμές των a και b που υπολογίσαμε και το r για το οποίο κάνουμε αυτούς τους υπολογισμούς. Σε περίπτωση που βρισκόμαστε εντός των ορίων του Hough πίνακα τότε το τρέχων σημείο ρίχνει εκεί τη ψήφο του, αυξάνοντας τη τιμή του κελιού του Hough κατά μια μονάδα.

Αφού ολοκληρωθεί η διαδικασία της "ψηφοφορίας", διατρέχουμε απ' όλα τα στοιχεία του πίνακα Hough ώστε να αποσπάσουμε τους πιθανούς κύκλους. Πιο συγκεκριμένα για κάθε κελί, εάν το πλήθος των ψήφων ξεπερνάει το όριο το οποίο έχουμε θέσει (*V_min*), τότε αυτός ο κύκλος μας ενδιαφέρει οπότε θέλουμε να υπολογίσουμε τις πιθανές συντεταγμένες του. Τόσο για τα a, b όσο και το r τις τιμές τις υπολογίζουμε ως τη μέση τιμή μεταξύ του κελιού που βρισκόμαστε και του επόμενου. Δηλαδή:

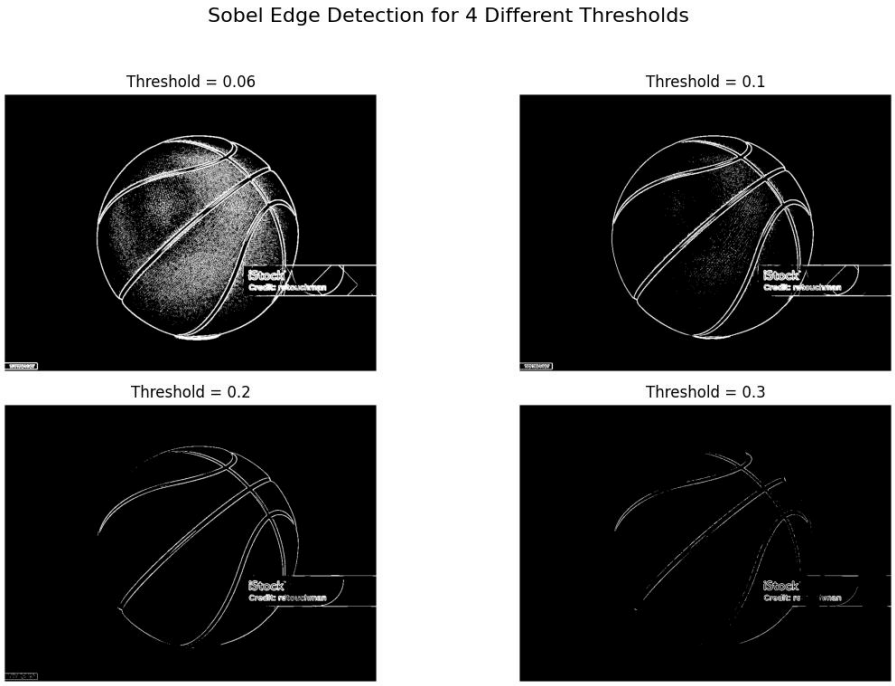
$$a = \frac{(a_i + a_{1+1})}{2} \quad b = \frac{(b_i + b_{1+1})}{2} \quad r = \frac{(r_i + r_{1+1})}{2}$$

4 Τελικά Αποτελέσματα και Συμπεράσματα

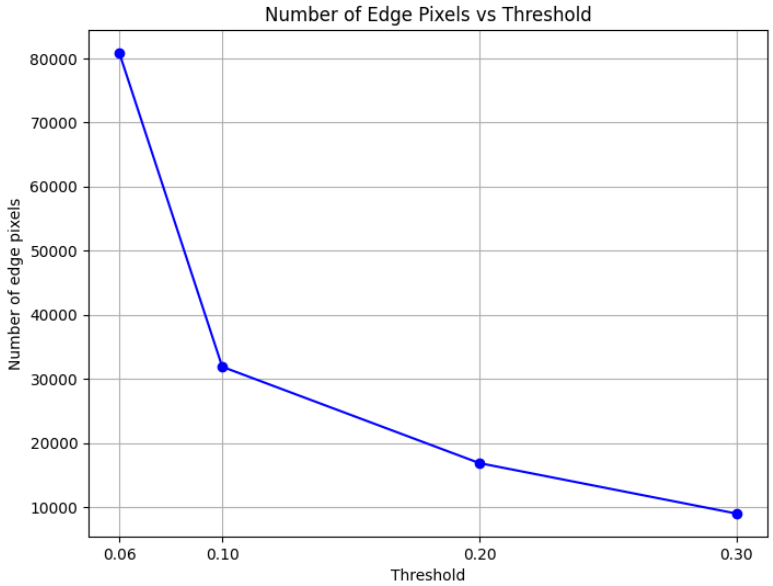
Σε αυτό το σημείο να προστεθεί πως όλα τα αποτελέσματα που θα ακολουθήσουν έχουν γίνει για μια downscaled έκδοση της αρχικής μας εικόνας και συγκεκριμένα οι νέες διαστάσεις είναι $(1001, 743)$.

4.1 Edge Detection

Αρχικά θα ασχοληθούμε τα αποτελέσματα της εφαρμογής του Sobel, παρακάτω παρατίθενται τα αποτελέσματα για εφαρμογή με 4 διαφορετικά thresholds $[0.06, 0.1, 0.2, 0.3]$.



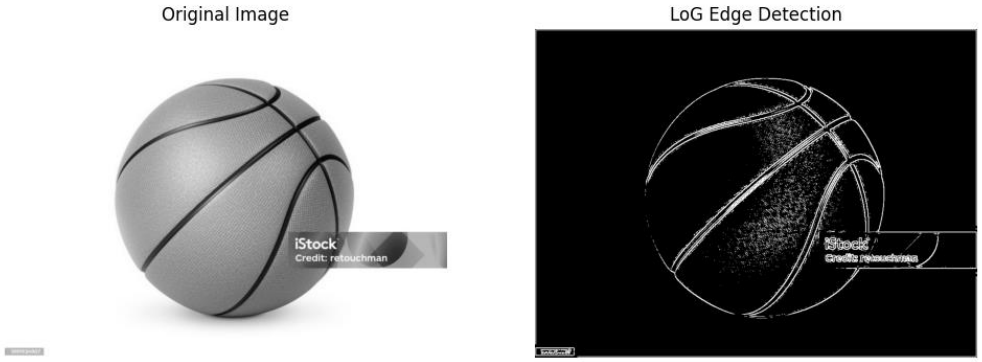
Εικόνα 1: Sobel Ανίχνευση Ακμών για διαφορετικές τιμές κατωφλίου



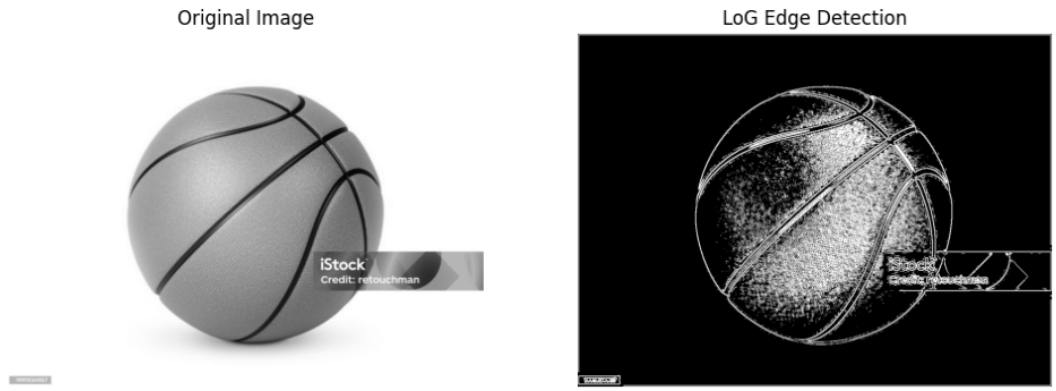
Εικόνα 2: Πλήθος ανιχνευόμενων σημείων ως ακμή σε συνάρτηση με το κατώφλι

Παρατηρούμε, τόσο από τις εικόνες εξόδου αλλά και κυρίως από τη γραφική παράσταση, πως όσο αυξάνεται η τιμή του threshold, η ποιότητα των αποτελεσμάτων του εντοπισμού ακμών μειώνεται. Αυτό είναι αναμενόμενο, καθώς με την αύξηση του threshold επιτρέπεται σε όλο και λιγότερα pixels να χαρακτηριστούν ως ακμές, με αποτέλεσμα να χάνονται λεπτομέρειες και να διατηρούνται μόνο οι πιο έντονες ακμές της εικόνας.

Στη συνέχεια, ακολουθούνε τα αποτελέσματα για την ανίχνευση ακμών έπειτα από την εφαρμογή LoG. Εδώ να προστεθεί το γεγονός πως τα συγκεκριμένα αποτελέσματα προέκυψαν έπειτα από επιλογή του threshold ίσο με 0.2 και 0.1.



Εικόνα 3: LoG Ανίχνευση Ακμών για threshold = 0.2



Εικόνα 4: LoG Ανίχνευση Ακμών για threshold = 0.1

Για τη πρώτη επιλογή του threshold = 0.2, μπορούμε να παρατηρήσουμε πως έχουν χαθεί αρκετές ακμές από το περίγραμμα της μπάλας, ιδιαίτερα στο αριστερό τμήμα της μπάλας και στη βάση της, καθώς εκεί δεν έχουμε και τόσο έντονες αλλαγές στα χρώματα (από ανοικτό γκρι σε λευκό και από σκούρο γκρι σε "ανοικτό" μαύρο αντίστοιχα). Από την άλλη, για τη δεύτερη επιλογή threshold = 0.1, βλέπουμε πως έχει ανιχνευθεί πιο επιτυχώς το περίγραμμα της μπάλας, όπως επίσης και περισσότερες λεπτομέρειες από το εσωτερικό της. Βέβαια, εάν παρατηρήσουμε πιο προσεκτικά θα δούμε πως συγκριτικά με τα αποτελέσματα του Sobel έχουμε πιο θορυβώδες αποτέλεσμα καθώς σε αρκετά σημεία εμφανίζονται pixels ακμής τα οποία όμως δεν είναι όντως στη πραγματικότητα.

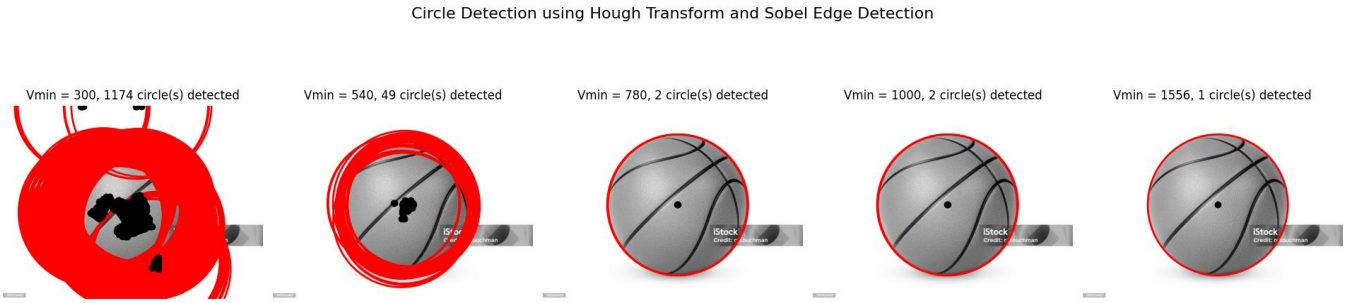
Ο Sobel από την άλλη για επιλογή threshold 0.1 και 0.06 , έχει καταφέρει να ανιχνεύσει επιτυχώς όλο το περίγραμμα της μπάλας ενώ για όλες τις επιλογές του threshold δεν παρατηρούμε ιδιαίτερη εμφάνιση θορύβου.

Όλος αυτός ο θόρυβος στην εφαρμογή του LoG θεωρώ πως οφείλεται στο τρόπο που εφάρμοσα το zero crossing. Λαμβάνοντας υπ' όψη πως βασική συνθήκη για να θεωρήσουμε ακμή είναι να υπάρχουν δυο γειτονικά pixels με τα οποία το κεντρικό να έχει διαφορά προσήμου, είναι πολύ πιθανό κάποιο pixel να θεωρηθεί ως ακμή, ενώ απλά πρόκειται για κάποιο θόρυβο.

4.2 Circle Detection

Τελευταίο ζητούμενο της εργασίας, ήταν η εφαρμογή του Hough Transform για την ανίχνευση κύκλων. Η συγκεκριμένη μέθοδος, όπως και είδαμε βασίζεται στην ανίχνευση ακμών οπότε αρχικά θα σχολιάσουμε τη εφαρμογή του Sobel Edge Detection και στη συνέχεια του Log Edge Detection.

Παρακάτω, μπορούμε να δούμε τα αποτελέσματα που βασίστηκαν στο Sobel Edge Detection:



Εικόνα 5: Sobel based Hough transform για Ανίχνευση Κύκλων

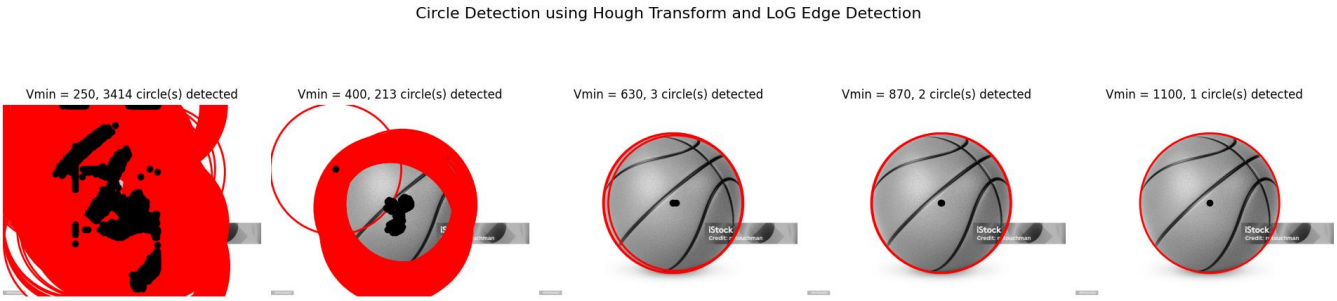
Η αρχική τιμή που δόθηκε για το *V_min* ήταν 300. Σε αυτήν την περίπτωση ανιχνεύτηκαν 1174 κύκλοι, αριθμός αρκετά μεγάλος. Οι κύκλοι αυτοί προκύπτουν όχι μόνο από το περίγραμμα της μπάλας, αλλά και από τις εσωτερικές γραμμές που αποτελούν μέρος του σχεδίου της.

Αυξάνοντας το *V_min* σε 540, οι ανιχνευόμενοι κύκλοι μειώνονται σημαντικά στους 49. Σε αυτήν την περίπτωση, οι περισσότεροι κύκλοι φαίνεται να σχετίζονται με το περίγραμμα της μπάλας, αλλά εντοπίζονται με διαφορετικές τιμές ακτίνας r, γεγονός που οδηγεί σε πολλαπλές επικαλυπτόμενες ανιχνεύσεις για τον ίδιο κύκλο.

Για τιμές *V_min* = 780 και *V_min* = 1000, ανιχνεύονται μόνο δύο κύκλοι, οι οποίοι έχουν ίδιες συντεταγμένες (a, b) και ακτίνα r.

Τέλος, με τιμή *V_min* = 1556, εντοπίζεται μόνο ένας κύκλος, ο οποίος αντιστοιχεί στο περίγραμμα της μπάλας. Η τιμή αυτή επιλέχθηκε έπειτα από παρατήρηση καθώς από τους δυο τελευταίους κύκλους, ο ένας είχε 1555 ψήφους ενώ ο άλλος 1560.

Τέλος, έχουμε τα αποτελέσματα που βασίστηκαν στο LoG Edge Detection και συγκεκριμένα για threshold = 0.1:



Εικόνα 6: Εικόνα 5: LoG based Hough transform για Ανίχνευση Κύκλων

Για να δούμε και διαφορετικά αποτελέσματα από τη προηγούμενη εφαρμογή, η αρχική τιμή που δόθηκε για το *V_min* ήταν χαμηλότερη και συγκεκριμένα 250, με αποτέλεσμα να ανιχνευθούν 3414 κύκλοι. Όπως και πριν, ο υπερβολικά μεγάλος αριθμός κύκλων οφείλεται στο ότι η πολύ χαμηλή τιμή του *V_min* επιτρέπει στο σύστημα να θεωρεί ως κύκλους οποιαδήποτε θορυβώδη ή έντονη περιοχή των ακμών, συμπεριλαμβανομένων εσωτερικών λεπτομερειών και αντιθέσεων που δημιουργούνται από τις γραμμές στη μπάλα αλλά και από το φόντο.

Αυξάνοντας την τιμή του *V_min* σε 400, ο αριθμός των κύκλων μειώνεται δραστικά σε 213. Αν και πάλι παρατηρείται υπερανίχνευση, οι κύκλοι πλέον συγκεντρώνονται περισσότερο γύρω από το περίγραμμα της μπάλας και λιγότερο στο εσωτερικό.

Για *V_min* = 630, παρατηρούμε πως ανιχνεύονται 3 κύκλοι, με δύο από αυτούς να είναι πιθανώς παρόμοιοι σε θέση και ακτίνα.

Στη συνέχεια, με τιμή *V_min* = 870, ανιχνεύονται 2 κύκλοι, οι οποίοι φαίνεται να είναι και πανομοιότυποι.

Τέλος, με τιμή *V_min* = 1100, ανιχνεύεται μόνο ένας κύκλος, ο οποίος αντιστοιχεί στο περίγραμμα της μπάλας. Και πάλι, Η τιμή αυτή επιλέχθηκε έπειτα από παρατήρηση καθώς από τους δυο τελευταίους κύκλους, ο ένας είχε 1086 ψήφους ενώ ο άλλος 1110.

Ως τελευταία παρατήρηση, μπορούμε να δούμε πως η ανίχνευση του κύκλου που βασίστηκε στη LoG ανίχνευση ακμών χρειάστηκε αρκετά λιγότερες ψήφους από αυτή που βασίστηκε στη Sobel ανίχνευση ακμών. Αυτό είναι ένα εύλογο αποτέλεσμα, καθώς όπως είχαμε τονίσει και στην ανάλυση των αποτελεσμάτων της LoG ανίχνευση ακμών, το περίγραμμα της μπάλας είχε κάποια “κενά”. Άρα λιγότερα pixel είχαν ανιχνευθεί ως ακμή δηλαδή λιγότεροι ψήφοι για τον κύκλο.